Module 2 – Introduction to Programming

THEORY EXERCISES

Theory Exercises

1. Overview of C Programming

THEORY EXERCISE:

- **Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.**

ANS: C programming was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It evolved from an earlier language called B, which itself was derived from BCPL. C was originally designed to write the Unix operating system, giving it a strong foundation in systems-level programming.

Throughout the 1980s, C gained widespread adoption as it offered a powerful combination of low-level memory control and high-level readability. In 1989, the American National Standards Institute (ANSI) standardized the language, producing what is known as ANSI C or C89, ensuring consistency across different platforms.

C's importance lies in its efficiency, portability, and direct hardware access. It serves as the foundation for many modern languages, including C++, Java, and Python, making it essential for understanding programming fundamentals.

Today, C remains widely used in embedded systems, operating systems like Linux and Windows, device drivers, and real-time applications where performance is critical. Its lightweight nature and close-to-hardware capabilities make it irreplaceable in fields where speed and resource management are priorities. Learning C builds a strong programming foundation that benefits developers across all disciplines.

2. Setting Up Environment

THEORY EXERCISE:

- **Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.**

ANS: Installing C Compiler:

GCC (GNU Compiler Collection) is one of the most widely used C compilers. The installation steps vary by operating system.

Setting up IDE:

Download Dev C++ from the official website (sourceforge.net). Run the installer and follow the on-screen steps. Dev C++ comes bundled with a MinGW compiler, so no separate compiler installation is needed. Once installed, go to Tools → Compiler Options to confirm the compiler path is correctly configured.

3. Basic Structure of a C Program

THEORY EXERCISE:

- **Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.**

ANS: Every C program consists of 5 key components:

1. Header Files: It's a set of libraries containing predefined data or also called as Built-in Libraries. It is called by writing #include<stdio.h> and so on.
2. Main Function: This part of program executes your code. It is called using main().
3. Comments: Comments are a part of program that helps a developer to understand what the code actually means and what it is being used for, this helps majorly when you are working on a group project and tasks are separated. They are called by writing // or /* */
4. Data Types: Data types are used to let the function know what type of values are being used, are they number(integers) or decimal number(float) or alphabet/special characters (char) and so on.
5. Variables; Variables are used to store a value. They must be declared before using. For ex. int c or float a. Important thing to remember variable can only start with a alphabet or underscore _. It can have numbers but after the condition is met.

4. Operators in C

THEORY EXERCISE:

- **Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.**

ANS: Operators in C:

1. Arithmetic: These are mathematical operators used to do addition, substraction, multiplication and so on. EX: +, -, %, *
2. Relational: These type of operators are used to compare two value. If they are same the value returned is 1 or else 0.\ EX: ==,<,>
3. Logical: These operators are used to combine two or more conditions. EX: &&, ||
4. Assignment: These operators are used to assign value to variables. EX =
5. Increment/decrement: These operators sole purpose is same as its name to increase and decrease its value. They are used as i++ or i—
6. Bitwise: These operators work directly on the binary level.
7. Conditional: These operators are used widely with loops to let the program know how long or how many times does the loop needs to run for. EX: ?, :

5. Control Flow Statements in C

THEORY EXERCISE:

- **Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.**

ANS: These are loops that helps the program to understand how to follow the code or how long does a specific condition needs to run for.

1. IF: This loop is used to check if the given condition is correct or not. If the condition is correct the program will keep running if not then it will exit.

   EX: i=3;

   ```
   If(i>3){
           //code }
   ```

2. Else: This is widely used with if loop to print a definite output. Like if you press 1 your are right or else wrong number.

   EX: i=3;

   ```
   If(i>3){ print hello }
   Else { print bye }
   ```

3. Nested if else: This is used when you want to give the program more than one condition to get satisfied result usually used for grading system.

   EX: if(m>50) { you are first }

   If (m>45) { you are second }

   Else { you failed }

4. Switch: This is also used to check more than one condition but the benefit of using switch is that it will check the first condition if it is correct then the program will exit the loop and provide the output unlike if which the checks the condition second time no matter what.

   EX: i=3;

   ```
   Switch(i>3){
       Case 1:
               i<3;
       break;
       case 2:
               i=4;
       break;
       default:
               wrong input.}
   ```

6. Looping in C

THEORY EXERCISE:

- **Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.**

ANS: Comparison.

- for loop best used when u know the umber of iteration in advance.
  EX: for (int i = 1; i <= 5; i++) { printf("%d\n", i);

- While loops: These loops are used when the iterations are unknown but you put a condition.
  EX: int i = 1; while (i <= 5) { printf("%d\n", i); i++; }

- Do-While loop: These loop is used when you want the program toe xecute at least no matter the condition.
  EX: int i = 1; do { printf("%d\n", i); i++; } while (i <= 5);

7. Loop Control Statements

THEORY EXERCISE:

- **Explain the use of break, continue, and goto statements in C. Provide examples of each.**

ANS:

- Break: This statement exits the loop as soon as the condition is met.
  EX: for (int i = 1; i <= 10; i++) { if (i == 5) { break; // stops loop when i reaches 5 } printf("%d\n", i); }

- Continue: This statement skips \the current iteration and move to the next one.
  EX: for (int i = 1; i <= 5; i++) { if (i == 3) { continue; // skips when i equals 3 } printf("%d\n", i);

- Goto: This statement lets you tell the program to goto to a specific part.
  EX: start: if (i <= 5) { printf("%d\n", i); i++; goto start;

8. Functions in C

THEORY EXERCISE:

- **What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.**

ANS: Functions are a part of the program that you can declare it once and keep using it as many time as you want throughout the program. It avoids repetition of the code. They should be declared before the main function. To declare a function write data_type function_name. To call a function just mention it in the main block of code.

9. Arrays in C

THEORY EXERCISE:

- **Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.**

ANS: Array is a collection of elements of same data type they are stored in the same memory address using an index. There are 2 types of arrays One dimensional and two dimensional. One dimensional array stores value in a single where as two dimensional stores value in rows and columns. For simple tasks one dimensional array is preferred where as for tables and matrix's two dimensional array is preferred.

One Dimesional Ex:

```
#include <stdio.h>
 int main() {
int marks[5] = {90, 85, 78, 92, 88};
for (int i = 0; i < 5; i++) {
printf("Student %d : %d\n", i+1, marks[i]); }
return 0; }
```

Two Dimensional EX:

```
#include <stdio.h>
int main() {
int marks[3][3] = { {90, 85, 78}, {88, 92, 95}, {70, 75, 80} };
 for (int i = 0; i < 3; i++) {
 for (int j = 0; j < 3; j++) {
printf("marks[%d][%d] = %d\n", i, j, marks[i][j]); }
}
return 0; }
```

10. Pointers in C

THEORY EXERCISE:

- **Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?**

ANS: Pointers are variables that stores the memory address of another variable instead of storing in a direct value. This helps developer to use the same variable value again in the program later. They are declared using * They are important because they allow direct memory access which makes programs faster.

11. Strings in C

THEORY EXERCISE:

- **Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.**

ANS: Strlen() is used to find the length of the string

Strcpy() is used to copy one string into another.

Strcat() is used to connect or merge two strings together.

Strcmp is used to compare two strings

Strchr() is used to search the first occurrence of a character in a string.

12. Structures in C

THEORY EXERCISE:

- **Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.**

ANS: A structure (struct) is a user defined data type that allows to store variables of different data type under a single name. To define a structure you use the keyword struct. You declare a stricter in the main body. To access a structure you have to use . with the variable. Ex s1.i

13. File Handling in C

THEORY EXERCISE:

- **Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.**

ANS: File handling allows a program to store data permanently on the hard drive so it can be accessed later. There types of file handling.

- Opening: fopen it lets you open a file in a read only mode.
- Closing: fclose lets you close the file to free up the memory. You must close the file.

  To read or modify a file there are different modes. r(read) lets you just read the data. W(write) lets you write new data. a(append) lets you modify current data.